Multiclass Image Classification Using CNN and Transfer Learning

Assignment 2

Course: Deep Learning Architectures and Techniques
Name: Diya Das
Roll No: 2501940036
Faculty: Dr. Shahid Ahmad Wani

Department of Computer Science and Engineering

School of Engineering and Technology

K.R Mangalam University, Gurugram- 122001, India

1. Introduction

Image classification is one of the most important applications of deep learning. Convolutional Neural Networks (CNNs) are widely used for extracting spatial features from images. In this assignment, a custom CNN model is implemented to classify images from the CIFAR-10 dataset. Additionally, transfer learning using a pretrained MobileNetV2 model is applied to compare performance. The objective is to understand convolution layers, pooling operations, and the effectiveness of pretrained models for multiclass image classification.

2. Dataset Description

The CIFAR-10 dataset is a widely used benchmark dataset for image classification tasks. It contains 60,000 color images divided into 10 different classes. Each image has a resolution of 32×32 pixels with three color channels (RGB). The dataset is split into 50,000 training images and 10,000 testing images.

The dataset includes the following classes:

Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck.

3. Data Preprocessing

Before training the neural network, several preprocessing steps were performed:

- Normalization: Pixel values were scaled from 0–255 to 0–1 to improve training stability.
- Train-Validation Split: The dataset was divided into training and validation sets.
- Label Encoding: Class labels were converted into categorical format using one-hot encoding.
- Data Augmentation: Techniques such as rotation, width shifting, height shifting, and horizontal flipping were applied to increase dataset diversity and reduce overfitting.

4. Custom CNN Architecture

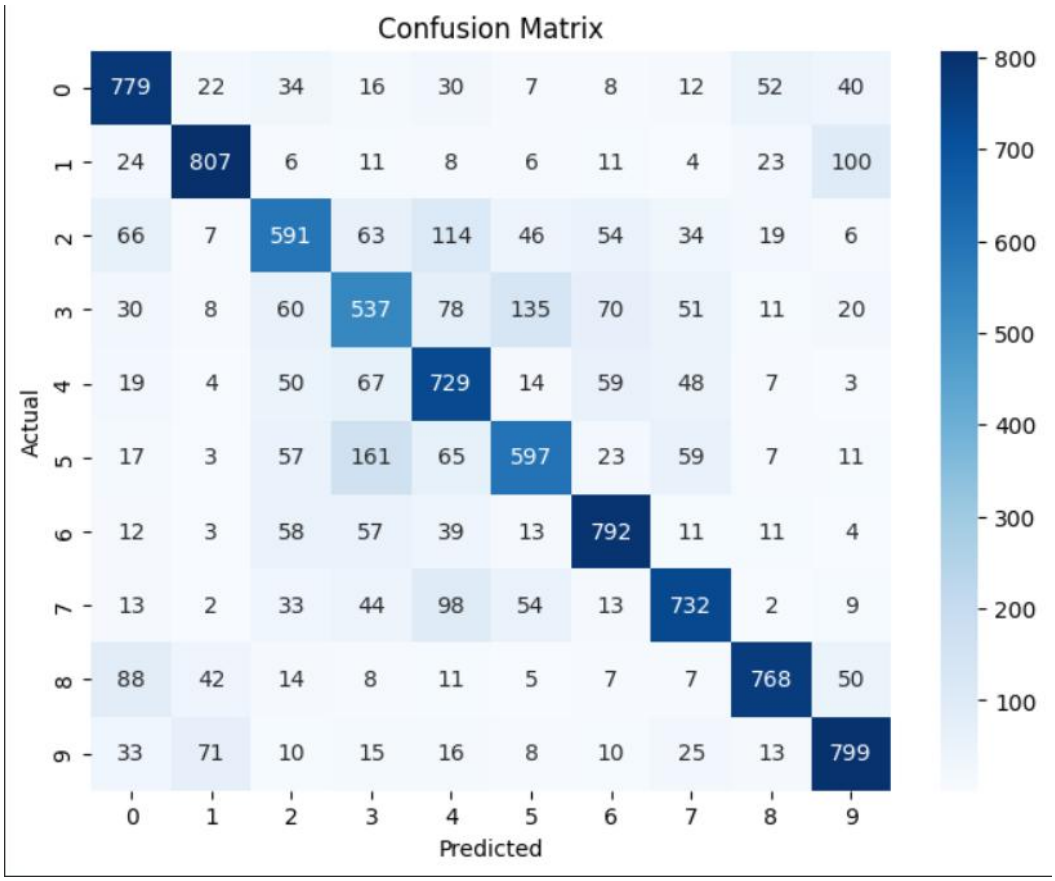
A custom Convolutional Neural Network (CNN) architecture was designed for multiclass image classification. The model consists of multiple convolutional layers for feature extraction, followed by pooling layers to reduce spatial dimensions. The extracted features are flattened and passed through fully connected layers to perform classification.

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_2 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_3 (Conv2D)	(None, 4, 4, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 2, 2, 128)	0
flatten (Flatten)	(None, 512)	0
dense (Dense)	(None, 128)	65,664
dense_1 (Dense)	(None, 10)	1,290

Total params: 160,202 (625.79 KB)
Trainable params: 160,202 (625.79 KB)
Non-trainable params: 0 (0.00 B)

5. Model Training

The CNN model was trained using the Adam optimizer and categorical cross-entropy loss function. The model was trained for 10 epochs using the training dataset, while validation data was used to monitor performance during training.



	precision	recall	f1-score	support
0	0.72	0.78	0.75	1000
1	0.83	0.81	0.82	1000
2	0.65	0.59	0.62	1000
3	0.55	0.54	0.54	1000
4	0.61	0.73	0.67	1000
5	0.67	0.60	0.63	1000
6	0.76	0.79	0.77	1000
7	0.74	0.73	0.74	1000
8	0.84	0.77	0.80	1000
9	0.77	0.80	0.78	1000
accuracy			0.71	10000
macro avg	0.71	0.71	0.71	10000
weighted avg	0.71	0.71	0.71	10000

7. Transfer Learning Using MobileNetV2

Transfer learning was implemented using the pretrained MobileNetV2 model. The pretrained convolutional layers were used as a feature extractor, while new dense layers were added for classification. The base model layers were frozen to retain pretrained weights while training only the new classifier layers.

Model: "sequential_2"

Layer (type)	Output Shape	Param #
mobilenetv2_1.00_224 (Functional)	(None, 1, 1, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dense_2 (Dense)	(None, 128)	163,968
dense_3 (Dense)	(None, 10)	1,290

Total params: 2,423,242 (9.24 MB)
Trainable params: 165,258 (645.54 KB)
Non-trainable params: 2,257,984 (8.61 MB)

8. Model Comparison

Model	Accuracy
Custom CNN	**0.71**
MobileNetV2	**0.355**

The custom CNN achieved better accuracy compared to the pretrained MobileNetV2 model. This occurred because CIFAR-10 images are very small (32×32), whereas MobileNetV2 is originally trained on larger images (224×224). Therefore, the pretrained features were not fully effective without resizing.

9. Conclusion

In this assignment, a custom CNN model and a transfer learning approach were implemented for multiclass image classification. The custom CNN successfully learned features from the CIFAR-10 dataset and achieved good classification performance. Transfer learning using MobileNetV2 was also explored to understand the benefits and limitations of pretrained models. This experiment helped in understanding convolutional layers, pooling operations, and the practical application of deep learning models for image classification tasks.